

Crypto Tracker & AI Hub — Complete Architecture, Design Story & Interview Guide

This document provides a comprehensive overview of the **Crypto Tracker & AI Hub** project, detailing its absolute system lifecycle story (with key terms defined in brackets), folder blueprints, configurations, routing parameters, dependencies reference, dictionaries for React and CSS concepts, JS methods, HTML tags, and libraries, and common technical interview questions with answers.

Table of Contents

- [1. The Story of the Crypto Tracker & AI Hub \(System Lifecycle\)](#)
- [2. Live AI Page \(Rubina\) Mechanics](#)
- [3. Folder & File Structure Details](#)
- [4. Local Installation & Setup Guide](#)
- [5. Firebase Security Rules & Database Schema](#)
- [6. React Core Concepts Dictionary](#)
- [7. CSS Styling Concepts Dictionary](#)
- [8. JavaScript Methods Dictionary](#)
- [9. HTML5 Tags Dictionary](#)
- [10. Libraries & APIs Directory](#)
- [11. Environment Variables Configuration](#)
- [12. Routing & Deployment Configuration](#)
- [13. Technical Interview Questions & Detailed Answers](#)

1. The Story of the Crypto Tracker & AI Hub (System Lifecycle)

Chapter 1: The Dev Environment & Webpack Bootup

Our story begins before the user even visits the website—in the developer's local environment.

When the developer runs `npm start`, Node.js starts executing **react-scripts** (a utility package from Create React App that manages build scripts, configurations, and hot-reload servers).

Because modern Node.js versions (v17+) use updated security algorithms that can crash older Webpack build tools, our startup script includes the **--openssl-legacy-provider** flag (a command-line argument that instructs Node.js to use legacy cryptographic algorithms to prevent build errors with older bundlers).

This initializes **Webpack** (a static module bundler that takes all JavaScript, CSS, and images and compiles them into optimized, browser-ready files). A **local development server** is created at `http://localhost:3000`, using a **Proxy Configuration** (a `package.json` setting that redirects local API requests, e.g. `/api`, to a separate backend server running at `http://localhost:5000`, avoiding CORS errors during testing).

Chapter 2: The First Visit & The Loading Gate

When the user types the URL, the browser retrieves our compiled index files.

Immediately, the application initializes our two global React contexts:

- ThemeContext.js** (a custom context provider that manages and distributes the active theme state, tracking whether the user prefers light or dark mode).
- CryptoContext.js** (a custom context provider that stores global app states like currency choice, symbol prefix, user details, watchlist array, and fetched coin list).

These contexts wrap the entire application in **index.js** using a **<Provider> pattern** (a design pattern in React where a component wraps around its children to make specific data and functions available to any child component below it in the tree).

At this initial second, the user **state** in `CryptoContext.js` is set to undefined (a JavaScript type representing a variable that has not yet been assigned a value, which we use to represent the "still checking session" state).

The main file, **App.js**, checks this state. While `user` is undefined, it displays a fullscreen golden loading spinner, preventing the app from showing a flashed preview of the login page or the dashboard.

Chapter 3: The Authentication Gate & Multi-Channel Alerting

Once the Firebase client completes token verification, it updates the **state**:

- If no active session exists, the **state** is set to null (a JavaScript value representing the intentional absence of any value, which we use to show the user is definitely logged out).
- This causes the app to block all main features and render the **LandingAuthPage.js** component.

This auth page uses a clean split-screen desktop layout:

- The Left Side:** Contains branding, a rotating logo, feature list, and colored ticker chips representing different cryptocurrencies.
- The Right Side:** Houses a glassmorphic authentication card containing a pill-style login/signup tab switcher, polished input fields, and a Google login button.

To manage tabs, we use **React State (useState)** (a React hook that allows components to maintain, read, and update local variable values that trigger a component re-render when changed).

When logging in:

- Email/Password Form:** The user inputs credentials. The form uses a **noValidate** attribute (an HTML attribute that disables the browser's default form validation dialogs, allowing custom JavaScript logic to handle all input checking). If inputs are valid, we call **Firebase Auth's** `signInWithEmailAndPassword` or `createUserWithEmailAndPassword`.
- Google OAuth Button:** Triggered via `signInWithPopup` (opens a separate authentication window for Google login). If the browser's popup blocker flags it, our catch block intercepts the error and immediately redirects the user using `signInWithRedirect` (redirects the current tab to Google's sign-in page and returns once completed).

When authentication succeeds, the **sendOwnerNotification** utility is executed. It fires three asynchronous notification alerts in parallel:

- EmailJS API** (a service that lets client-side JavaScript send automated emails directly via SMTP templates without a backend mail server).
- Discord Webhook POST Request** (an HTTP request that sends structured data embeds to a specific channel URL on Discord).
- Telegram Bot API Message** (a request sent to Telegram servers to dispatch a notification message to a specific chat ID via a bot).

Chapter 4: Single Page Navigation & The Header

With the user state now updated to a valid User Object, the landing page disappears, and the user enters the main application.

Page navigation is controlled by **react-router-dom** (the standard routing library for React that dynamically swaps page components in the browser without reloading the physical HTML page). We define our paths using:

- <BrowserRouter>** (a router component that uses the HTML5 history API to keep the UI in sync with the browser URL).
- <Route>** (a component that renders a specific page when the URL matches its path pattern).
- <Switch>** (a router component that renders only the first matching Route it encounters, preventing multiple pages from rendering at once).

At the top of the app is **Header.js** (a persistent global navigation component). It contains:

- Logo & Title:** Clicking it redirects the user to the **HomePage** / using a Router **<Link> component** (a React component that enables declarative, client-side navigation without triggering full page reloads).
- Currency Dropdown:** Allows the user to select between USD and INR. When changed, it updates the context state `currency`, which automatically updates `symbol` to \$ or ₹.

- 3. **Theme Switch Icon:** Connects to the `ThemeContext` and runs `toggleTheme()`, swapping styles from dark mode `#0f1117` to light mode `#ffffff`.
- 4. **User Avatar:** An icon representing the logged-in user. Clicking it opens the `UserSidebar.js` component, which is implemented as a **Drawer** (a Material-UI component that slides out from the edge of the screen like a sidebar drawer).

Chapter 5: The Market Dashboard & Search Table

On the `HomePage (/)`, the user sees two major features:

1. The Trending Carousel

At the top is a horizontal, sliding carousel showing trending coins. It fetches trending data from CoinGecko, maps them to animated slides, and mounts them inside the `react-alice-carousel` component (a responsive, touch-friendly React carousel package that supports dragging, infinite looping, and autoplay animations).

2. The Interactive Coin Table

Below the carousel is the list of 100+ cryptocurrencies. We use a search bar to filter coins. This uses a real-time search handler:

```
coins.filter((coin) =>
  coin.name.toLowerCase().includes(search.toLowerCase()))
)
```

- `.filter()` (a JavaScript array method that creates a new array containing only elements that pass a test condition).
- `toLowerCase()` (a JavaScript method that converts string characters to lowercase, ensuring searches are case-insensitive).

To prevent the user from being overwhelmed by a massive list, the table implements **Pagination** (a UI pattern that splits large datasets into pages, rendering only 10 rows per page using Material-UI's Pagination component).

Chapter 6: Historical Charts & Optional Chaining

Clicking a coin in the table opens `CoinPage.js` (the dynamic coin details page mapped to `/coins/:id`). The router uses a `useParams` hook (a React Router hook that extracts dynamic URL parameters, like the coin's unique ID, directly from the address bar).

This page uses **Axios** (a promise-based HTTP client for making API requests) to fetch detailed market capitalization, descriptions, and current rank.

Since data loading takes a split second, rendering properties of a null object would crash the app. We prevent this using **Optional Chaining (?.)** (a JavaScript operator that reads nested object fields safely, returning undefined instead of throwing a fatal crash error if the object is missing):

```
coin?.market_data?.current_price[currency.toLowerCase()]
```

In the center of the page is the historical chart, rendered by `CoinInfo.js`. It fetches pricing points based on the active timeframe (24h, 30d, 1y) selected via custom `SelectButton.js` components.

The chart data is rendered using a **<Line> Chart component** from `react-chartjs-2` (a React wrapper for Chart.js).

- **Optimized Rendering:** The chart uses `pointRadius: 0` (disables rendering dots for every coordinate to save CPU rendering cycles) and only displays dots on hover.
- **Tension:** Set to `0.25` (a Chart.js setting that smooths the chart path into curves rather than sharp, jagged lines).

Chapter 7: The Real-Time Watchlist Sync

If the user wants to bookmark the coin, they click the favorite button on `CoinPage.js`.

We point to the database:

```
const coinRef = doc(db, "watchlist", user.uid);
```

- `doc()` (a Firebase Firestore function that defines a reference to a document in a collection using its path).
- `db` (the initialized Firestore database instance we import from `firebaseConfig.js`).

The app uses `async/await` (JavaScript keywords used to pause execution inside an async function until a Promise completes, making async code run in a structured, line-by-line format) to write the change to **Cloud Firestore** (Google's real-time, cloud-hosted NoSQL database).

We save the array using the **Spread Operator (...)** (a JavaScript operator that expands an array, copying existing items into a new list while appending a new item at the end):

```
{ coins: watchlist ? [...watchlist, coin?.id] : [coin?.id] }
```

We pass `{ merge: true }` (a database parameter that merges new data fields into an existing document instead of overwriting the whole document).

Immediately, a listener in `CryptoContext.js` called `onSnapshot` (a Firestore database listener that opens a persistent connection and fires a callback function whenever the targeted document changes) detects this database update. It updates the global watchlist state, causing the favorite button and the sidebar list in `UserSidebar.js` to update instantly.

To delete a coin, we call the same reference, but use `.filter()` (an array method that returns a new array with specific items removed) to filter out the coin's ID, updating the document in Firestore.

Chapter 8: Secure AI Conversations

For market insights, the user opens the chatbot to talk to "Rubina". When a prompt is typed, the React app makes a POST request to `/api/ai`, which is a **Vercel Serverless Function** (a backend script hosted in the cloud that executes on-demand when called, rather than running continuously on a server VM).

We use a serverless function to protect the **GEMINI_API_KEY** (a secret authorization token used to securely identify and authenticate our requests to Google's AI servers). If this key were in React, anyone could open their browser console and steal it.

When the request hits `/api/ai`:

1. **CORS Headers** (Cross-Origin Resource Sharing headers, which restrict backend access only to authorized client domains like localhost or your Vercel deployment URL) are checked.
2. **Rate Limiting** (a security countermeasure that blocks requests from an IP address if it exceeds 20 requests per minute, preventing spam attacks) checks the client's IP.
3. **Prompt Injection Scanner** (a script that scans the input text for malicious jailbreak phrases, e.g. "ignore previous instructions", and rejects them) validates the prompt.
4. **Google Gemini API** (Google's Generative AI service, specifically using the `gemini-2.5-flash` model) is queried securely. The response is returned to React and displayed in the chat.

Chapter 9: Deployment & Global Hosting

Once the project is complete, the developer pushes the code to **GitHub** (a cloud Git repository platform used for hosting code and tracking changes).

Vercel (a cloud platform optimized for deploying frontend applications and serverless backend handlers) detects the new commit. It runs a Node.js build process, verifies environment variables, bundles the React code, and deploys the entire application globally on secure, high-speed CDN edges!

2. Live AI Page (Rubina) Mechanics

The **Rubina AI Dashboard** (implemented inside `src/Pages/AiPage.js`) handles data-driven analysis and chat querying interfaces through three specialized layouts:

1. The Market Scanner (Scoring Algorithm)

The market scanner performs a math-based evaluation on top-ranked tokens. It pulls the current prices list, filters the top 50 coins, and calculates a custom score for each based on three parameters:

- **Momentum (50% weight):** Clamped 24h price change percentage:
momentum = clamp(change_24h / 15, -1, 1)
- **Liquidity (20% weight):** Volume vs market cap ratio:
liquidity = clamp((volume / market_cap) / 0.25, 0, 1)
- **Size Factor (30% weight):** Prefers established coins:
size = 1 - clamp(rank / 100, 0, 1)

Score Formula: Score = (Momentum * 0.5) + (Liquidity * 0.2) + (Size * 0.3). The app highlights the asset with the highest score as the recommended buy candidate alongside a percentage confidence rating.

2. Asset Summary (Fundamental Dossiers)

The Asset Summary formats current price, volume changes, daily high/low levels, and market cap rank into a readable paragraph summary, allowing users to quickly assess trading ranges.

3. General Query Interface (Gemini Integration)

Allows natural language questions about halving cycles, blockchain structures, or trading metrics. It communicates securely with Vercel's /api/ai endpoint.

3. Folder & File Structure Details

```
crypto-tracker/
├── api/
│   ├── ai.js           # Vercel Serverless API Functions (Node.js)
│   ├── health.js      # Secure Gemini Chat API endpoint
│   └── health.js      # Serverless Health status check
├── public/
├── server/
│   ├── ai-server.js   # Static Assets (favicon, icons, html wrapper)
│   └── ai-server.js   # Local Express Backend testing script
├── src/
│   ├── components/    # Local testing Node proxy server
│   │   ├── components/ # Frontend Source Code
│   │   │   ├── Authentication/ # Reusable React UI widgets
│   │   │   │   ├── Authentication/ # Authentication UI (Login/Signup / Sidebar)
│   │   │   │   ├── CoinsTable.js  # Main search data table
│   │   │   │   ├── CoinInfo.js    # Chart.js visualization component
│   │   │   │   └── Header.js      # Persistent Nav block
│   │   └── config/          # Constant configurations (APIs, static lists)
│   │       ├── api.js       # CoinGecko query endpoints configuration
│   │       └── firebaseConfig.js # Client credentials for Firebase setup
│   ├── Pages/           # Parent Page routes
│   │   ├── HomePage.js  # Market dashboard parent page
│   │   └── CoinPage.js  # Individual coin details parent page
│   ├── App.js           # Root Router & Security Gate Controller
│   ├── CryptoContext.js # Shared Global Market & User state provider
│   ├── ThemeContext.js  # Custom theme state controller (Dark/Light)
│   └── index.js          # Entrypoint bootstrap script
├── package.json
└── vercel.json          # Package manifests, scripts, engine variables
                        # Vercel Serverless Routing configurations
```

4. Local Installation & Setup Guide

1. **Prerequisites:** Install Node.js (v18 or higher) and npm.
2. **Install dependencies:** Run npm install in the root folder.
3. **Configure Local variables:** Create a .env file in the root directory:

```
GEMINI_API_KEY=your_gemini_api_key_here
REACT_APP_AI_ENDPOINT=http://localhost:3000/api/ai
REACT_APP_OWNER_NOTIFICATION_EMAIL=abhisarmawork07@gmail.com
REACT_APP_EMAILJS_SERVICE_ID=your_emailjs_service_id
REACT_APP_EMAILJS_TEMPLATE_ID=your_emailjs_template_id
REACT_APP_EMAILJS_PUBLIC_KEY=your_emailjs_public_key
```

4. **Start local serverless APIs:** Install Vercel CLI (npm install -g vercel) and run vercel dev.
5. **Build for production:** Run npm run build.

5. Firebase Security Rules & Database Schema

Firestore watchlist rules are configured to restrict read/write access strictly to the document owner matching their auth UID:

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /watchlist/{userId} {
      match /watchlist/{userId} {
        allow read, write: if request.auth != null && request.auth.uid == userId;
      }
    }
    match /signups/{userId} {
      allow create: if request.auth != null && request.auth.uid == userId;
      allow read, update: if request.auth != null && request.auth.uid == userId;
    }
  }
}
```

6. React Core Concepts Dictionary

Concept	What It Is (Definition)	Where & Why We Use It
Components & Props	Components are reusable, isolated UI building blocks. Props are read-only properties passed from parent to child to configure UI.	Used to modularize files. E.g. SelectButton.js receives a selected boolean prop to highlight itself.
useState Hook	A function that lets functional React components declare, read, and update local reactive state variables.	Used to track form email/password strings, AI loading animations, search terms, and current page numbers.
useEffect Hook	A function that lets components execute side effects (such as fetching data or setting up database listeners) when states change.	Used to trigger database synchronizations on login, query coin lists from CoinGecko, and update charts.
useContext Hook	A function that lets components subscribe directly to a React Context Provider, avoiding intermediate prop-drilling.	Used through the custom CryptoState() hook to access currency configurations, alerts, and user watchlists anywhere.
useMemo Hook	A function that caches (memoizes) the computed output of an expensive mathematical calculation between renders.	Used inside AiPage.js to compute and store the selected coin reference only when the selected dropdown ID changes.
useParams Hook	A React Router hook that extracts the dynamic parameters of the current URL matches (e.g. coin IDs).	Used inside CoinPage.js to read the coin's ID from the URL path (e.g., /coins/bitcoin) for details querying.

Conditional Rendering	A React practice that uses JavaScript logic (like ternaries or logical ANDs) to selectively display specific UI segments.	Used to display a loading spinner while <code>user === undefined</code> , and swap between dashboard access and the login card.
-----------------------	---	---

7. CSS Styling Concepts Dictionary

CSS Concept	What It Is (Definition)	How & Why We Use It
Flexbox	A one-dimensional layout module that manages alignment, sizing, and ordering of child elements inside a row or column.	Centering elements inside the login page, positioning the Google OAuth button, and arranging coin table layouts.
CSS Grid Layout	A two-dimensional grid system that structures layouts into responsive columns and rows.	Structuring the multiple tool cards (Market Scanner, Asset Summary, Query Interface) into columns on the AI Page.
CSS Keyframes (@keyframes)	A styling method used to define animations by changing style rules at specific percentage steps (from 0% to 100%).	Creates the <code>rotateLogo</code> keyframes that spin the brand and login cards logos continuously on a smooth loop.
Media Queries (@media)	Conditional CSS rules that apply specific styling properties only when device screens match specific width/height queries.	Hiding the left hero information panel on mobile screen queries to keep the login flow accessible on mobile viewports.
Material-UI JSS (makeStyles)	A CSS-in-JS solution that allows developers to write CSS declarations as JavaScript objects scoped locally to a component.	Powers all component-level styling, allowing styles to react dynamically to theme variables (light/dark mode toggle).
Glassmorphism	A user interface design trend characterized by translucent panels, subtle borders, and background-blur filters.	Applied to the right-side login card to create a premium, translucent container that glows against the dark background.
Linear & Radial Gradients	Visual transition gradients blending multiple colors smoothly along a straight line (linear) or out from a point (radial).	Renders the background of the left info panel and the soft gold glow background behind the auth card.

8. JavaScript Methods Dictionary

Method	What It Is (Definition)	Where & Why We Use It
<code>.map()</code>	Loops through an array, processes each element, and returns a new array of the same length.	Used to render lists of coins into HTML components (e.g., rendering table rows in <code>CoinsTable.js</code> or slides in the Carousel).
<code>.filter()</code>	Loops through an array and returns a new array containing only the elements that match a specific condition.	Used for filtering coin searches by matching name search strings, and for removing coins from the watchlist array.
<code>.includes()</code>	Checks if a specific value exists inside an array or string, returning a boolean (true/false).	Used to check if a specific cryptocurrency ID is currently saved inside the user's watchlist array.
<code>.find()</code>	Searches an array and returns the very first element that matches the provided test function.	Used on the AI page to find the specific coin object corresponding to the user's selected dropdown ID.
<code>.sort()</code>	Sorts the elements of an array in place based on a comparison function and returns the sorted array.	Used in the AI Market Scanner to rank coins from highest to lowest calculated buy score.
<code>.slice()</code>	Extracts a section of an array and returns it as a new array without modifying the original data.	Used to grab only the top 50 or top 100 coins for display, improving page layout performance.
<code>.toLowerCase()</code> / <code>.toUpperCase()</code>	Converts all characters in a string to lowercase or uppercase respectively.	Used to normalize search queries for case-insensitive matching and to format coin tickers (e.g. BTC).
<code>JSON.stringify()</code> / <code>JSON.parse()</code>	Stringify converts JavaScript objects to JSON strings; Parse converts JSON strings back into JavaScript objects.	Used to serialize and deserialize data stored inside <code>localStorage</code> cache to avoid network limits.
<code>preventDefault()</code>	Cancels the browser's default action associated with an event (like refreshing the page during form submit).	Used in form submit handlers in <code>LandingAuthPage.js</code> to allow customized JS validation to process credentials first.
<code>Math.min()</code> / <code>Math.max()</code>	Returns the lowest or highest number from a list of numeric arguments respectively.	Used inside the custom <code>clamp()</code> function to keep calculated AI scoring values within the 0 to 1 boundaries.

9. HTML5 Tags Dictionary

Tag Name	What It Is (Definition)	How & Why We Use It
<code><div></code>	Generic division container used for grouping elements and applying CSS styles.	Used throughout the app to separate cards, structure grid layouts, and isolate component wrapper sections.
<code></code>	Inline container used to wrap small chunks of text or elements without breaking lines.	Used to style specific inline words (like coloring the word "crypto" gold inside headings).
<code><form></code>	Container tag that defines a document section containing interactive controls to submit information.	Wraps the email/password text fields on the login page to enable native form submission events.
<code><input></code>	Interactive input field used to collect user inputs.	Renders text areas for emails, passwords, and chatbot prompts. Wrapped by Material-UI's components.
<code></code>	Image element used to embed graphics and photos in the webpage.	Embeds the rotating app logo and the cryptocurrency icons on the dashboard.
<code><button></code>	Clickable UI element used to trigger actions or submit forms.	Creates clickable triggers for submitting search forms, running scans, and submitting AI queries.
<code><table></code> , <code><tr></code> , <code><th></code> , <code><td></code>	Element tags used to structure information into grid-style rows, headers, and cells.	Structures the 100+ cryptocurrencies list on the homepage into clean, sortable columns.
<code><pre></code> & <code><code></code>	Preformatted text container that preserves whitespace; code wraps code blocks.	Used in the documentation templates to render layout files, database schemas, and folder structures.

10. Libraries & APIs Directory

Library / API	What It Is (Definition)	Why We Use It In the Project
React.js	A front-end JavaScript library for building component-driven UI views.	Forms the foundation of the frontend, managing view components, local state, and global contexts.
Firebase SDK	A set of tools for integrating Google Cloud services into applications.	Powers user authentication (Email/Google) and syncs database watchlists in real-time.
Chart.js & react-chartjs-2	Chart.js is a canvas plotting library; react-chartjs-2 wraps it as React components.	Plots the interactive historical line charts for the selected coin pages.
Material-UI (v4)	A React UI component library implementing Google's Material Design guidelines.	Provides consistent pre-designed layouts, snackbars, dropdown selectors, side drawers, and fonts.
Axios	A promise-based HTTP client for running requests in browsers and Node.js.	Fetches historical currency prices and coin detail configurations from external APIs.
react-alice-carousel	A responsive slider wrapper component for React.	Renders the moving trending coins banner at the top of the dashboard.
Google Gemini API	Google's generative AI model suite (accessed via gemini-2.5-flash).	Processes and answers queries submitted by users in the "Rubina" chatbot dashboard.
CoinGecko API	A free, public cryptocurrency market data database.	Provides price feeds, historical ranges, and coin metadata to populate the dashboard.
EmailJS	A library that connects email services to client code directly.	Dispatches automated alerts to the developer's email whenever a signup/login occurs.

11. Environment Variables Configuration

Variable Name	Description	Used In
GEMINI_API_KEY	Private API Key to query Google's Gemini AI models. Must never be exposed in React.	<code>api/ai.js</code> (Server-only)
GEMINI_MODEL	Determines the target model (defaults to <code>gemini-2.5-flash</code>).	<code>api/ai.js</code> (Server-only)
ALLOWED_ORIGINS	Restricts CORS domains allowed to make POST calls to our APIs.	<code>api/ai.js</code> (Server-only)
REACT_APP_AI_ENDPOINT	Endpoint URL to query the assistant (e.g. <code>/api/ai</code> in production).	<code>src/components/Authentication/UserSidebar.js</code>
REACT_APP_OWNER_NOTIFICATION_EMAIL	Target email address for owner notifications (configured to <code>abhisharmawork07@gmail.com</code>).	<code>src/utlils/emailjs.js</code>
REACT_APP_EMAILJS_SERVICE_ID	EmailJS account service ID hook	<code>src/utlils/emailjs.js</code>
REACT_APP_EMAILJS_TEMPLATE_ID	EmailJS template ID representing email layout.	<code>src/utlils/emailjs.js</code>

12. Routing & Deployment Configuration

Below is the Vercel deployment structure defined in `vercel.json`:

```
{
  "version": 2,
  "build": {
    "env": {
      "NODE_VERSION": "22"
    }
  },
  "buildCommand": "npm run build",
  "outputDirectory": "build",
  "framework": "create-react-app",
  "rewrites": [
    {
      "source": "/api/(.*)",
      "destination": "/api/$1"
    },
    {
      "source": "/__auth/(.*)",
      "destination": "https://crypto-tracker-96ae3.firebaseio.com/__auth/$1"
    },
    {
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

13. Technical Interview Questions & Detailed Answers

Q1: Why did you initialize the user state as undefined instead of null in your React Context?

Answer:

"Initializing user state as null by default introduces a race condition. When the page first loads, the Firebase SDK takes a few hundred milliseconds to verify if a user has an active session token. If we start with null, the app assumes the user is logged out and immediately flashes the login landing page. By starting as undefined, we represent the 'authenticating' state. We display a loading spinner while the auth handler runs, and render the auth page only if the state resolves to null (confirmed logged out), or the dashboard if it resolves to a User object."

Q2: What is the benefit of using Vercel Serverless functions instead of hosting a standalone Express server?

Answer:

"Using serverless functions saves maintenance and infrastructure costs. We don't have to manage a continuously running server VM. Vercel hosts them as individual, on-demand microservices that scale automatically. They execute only when a user interacts with the chatbot, keeping operating costs low and deployment simple. Furthermore, it integrates directly with our deployment workflow without needing separate dev pipelines."

Q3: How do you handle rate-limiting in a serverless environment where instances might scale up or spin down?

Answer:

"In our application, we implement an in-memory Map rate limiter that checks client IPs. In a serverless environment, this is highly effective against basic user spam during a cold start or within a single instance. For enterprise-scale production where serverless instances scale horizontally and do not share memory, the best approach would be integrating a serverless-friendly database like Upstash (Redis) or Memcached. This ensures a centralized rate-limit key store across all parallel instances."

Q4: If a user's browser blocks popups, how does your Google Authentication handle it?

Answer:

"We implement a try-catch fallback pattern. When the user clicks 'Continue with Google', we fire `signInWithPopup(auth, provider)`. If the browser blocks it, Firebase throws an error with the code `auth/popup-blocked`. Inside our catch block, we check for this specific error code and immediately call `signInWithRedirect(auth, provider)`. The user is seamlessly redirected to the Google login page and returns to the app once authenticated."

Q5: How did you optimize React rendering performance when updating real-time crypto lists?

Answer:

"We utilize React's component state optimization and minimize DOM updates:

1. We keep our data arrays flat.
2. We use React's `key` attribute on map iterators (using unique coin IDs rather than index values) so React can surgically update changed items instead of re-rendering the whole list.
3. We use localized styling and debounced inputs in search bars so typing updates the UI instantly without triggering updates across parent containers."

Q6: Why did you choose React Context API over Redux for this application?

Answer:

"We chose not to use Redux because the application is medium-sized and does not require complex state mutations. Using Redux would have introduced a lot of boilerplate code (actions, reducers, store configuration, selectors). Instead, React Context API allowed us to share global states (like the selected currency, the authenticated user object, the watchlist array, and dark/light theme settings) throughout the app with a lightweight, clean, and built-in React feature."

Q7: How does your AI Market Scanner calculate buy scores for coins?

Answer:

"The Market Scanner uses a mathematical scoring model in `AiPage.js` that evaluates the top 50 assets based on three metrics. First, momentum is calculated from 24h price change percentage and clamped between -1 and 1. Second, liquidity is calculated by dividing 24h trading volume by market capitalization, representing active trading interest. Third, size factor is calculated inversely from the coin's market cap rank. These three parameters are weighted (50% momentum, 20% liquidity, and 30% size) to generate a final normalized buy confidence percentage."